

本章阅读导航

先抓住一条主线：DAC 允许只表示传统权限这一层通过；SELinux 还会用进程 domain、目标 type、对象类别和请求权限重新判断。调查时先取得模式和上下文，再用 AVC 把拒绝还原成可验证的访问请求。

- 01区分访问控制层确认 DAC 与 MAC 各自回答什么
- 02识别安全上下文从四段字段找到 domain 与 type
- 03分开当前与持久模式Enforcing、Permissive、Disabled 不混写
- 04建立主体与目标证据用 ps -Z 与 ls -Z 对应真实对象
- 05逐字段读取 AVC提炼 scontext、tcontext、tclass 与权限
- 06先分类再修复把证据交给文件、端口或 Boolean 入口

专题地图

*知识专题*DAC 允许之后，SELinux 怎样判断一次访问

*知识专题*从四段安全上下文识别 domain 与 type

*操作专题*识别当前模式、持久模式与策略状态

*操作专题*用 ps -Z 与 ls -Z 建立主体和目标证据

*知识专题*把一条 AVC 还原成一次访问请求

*操作专题*重新触发、检索并解释近期拒绝

*诊断专题*先证据后修复，并正确使用 Permissive

*经典任务*完成自定义 Web 目录 AVC 取证

*参考解答*从基线、复现到 AVC 分类

*本章收束*把拒绝写成可交接的判断句

阅读时持续回答

1. 普通权限允许后，SELinux 还会检查什么？
2. 进程 domain 和文件 type 从哪里观察？
3. 当前模式与持久模式是否一致？
4. Permissive 成功到底证明了什么？
5. AVC 中谁是主体、谁是目标、目标是什么类别？
6. 没查到 AVC 时，下一条证据是什么？
7. audit2why 能解释什么，不能替你决定什么？
8. 哪些修复必须留给第 29 章？

阅读方法：先区分访问控制层，再核对模式与上下文；只有取得 AVC 后，才把拒绝分类到下一步配置入口。

第 28 章 SELinux 模型、模式、上下文与 AVC 证据

一个服务报告 `Permission denied` 时，最危险的处理方式不是命令写错，而是把所有“拒绝”都看成同一件事。Linux 传统权限可能拒绝，服务自身配置可能拒绝，网络链路可能拒绝，SELinux 也可能在普通权限允许之后继续拒绝。若不先区分证据层，管理员很容易通过放宽权限、切换到 `Permissive`，甚至关闭 SELinux，让症状暂时消失，却没有真正解释系统为什么拒绝访问。

本章解释 SELinux 调查中的第一组核心对象：策略在判断谁访问谁，系统当前以什么模式执行策略，进程和对象当前带有什么上下文，以及一条 AVC 记录能证明什么。主线是“访问模型 → 模式状态 → 上下文证据 → AVC 解读 → 诊断分类”。文件路径的持久类型规则、端口类型和 Boolean 的完整修复操作留到下一章《SELinux 文件规则、端口类型与 Boolean》；`firewalld` 仍属于第 21 章。

概念

DAC 与 MAC 自主访问控制（DAC）根据 Linux UID、GID、权限位和 ACL 判断访问；SELinux 提供额外的强制访问控制（MAC）。DAC 拒绝时，SELinux 不会替它放行；DAC 允许也只能证明传统权限这一层通过，不能证明策略会允许。``ls -l``、``namei -l`` 与 ``getfacl`` 观察 DAC，``ps -Z``、``ls -Z`` 和 AVC 才进入 SELinux 证据层。

概念

Policy SELinux policy 是 domain、type、对象类别、权限、转换和约束的规则集合。它不是 Apache 配置，也不是 `firewalld` 规则；应用把目录或端口改到新位置，不会自动修改 SELinux policy。``sestatus`` 能观察已加载策略名，但具体访问结论仍要由上下文与 AVC 证明。

概念

Domain 进程安全上下文第三段的 type 通常称为 domain，例如 ``httpd_t``。domain 是策略眼中的访问主体身份，不等同于命令名或 Linux 用户；两个都叫 ``httpd`` 的进程也应 ``ps -Z`` 显示的当前上下文为准。

概念

Type 文件、目录、端口和其他对象的安全上下文第三段描述策略用途，例如 ``httpd_sys_content_t`` 或 ``var_t``。文件名和路径字符串不能替代 type；当前 type 可由 ``ls -Z`` 观察，但当前标签正确也不能自动证明持久文件规则存在。

概念

安全上下文 常见形式是 `user:role:type:level`。四段都属于上下文，但 RHCSA 服务故障最常从第三段开始：进程的 type 作为 source domain，目标对象的 type 作为 target type。第四段可能包含 MCS 类别，不能因为其中出现额外冒号就误拆字段。

概念

Enforcing / Permissive / Disabled Enforcing 加载策略并执行拒绝；Permissive 仍加载策略、维护标签并记录本应发生的拒绝，只是不执行拒绝；Disabled 不进入正常策略判断链。Permissive 是受控诊断变量，不是最终修复；当前模式与 `/etc/selinux/config` 中的持久计划必须分开验证。

概念

AVC Access Vector Cache 相关审计记录把一次拒绝还原为“主体 domain 对目标 type 的某类对象请求某项权限”。初步阅读优先找 `{ permission }`、`scontext`、`tcontext` 和 `tclass`，再用 PID、命令名、路径和事件序号关联业务动作。AVC 是证据，不是自动修复指令。

操作语义以下六组入口分别回答模式、临时切换、上下文、Audit 检索、辅助解释和 Journal 旁证。先理解命令作用对象，再记关键形式。

getenforce / sestatus

SYNOPSIS

```
getenforce
sestatus [-v] [-b]
```

读取当前 SELinux 模式和状态总览。`getenforce` 只回答当前模式；`sestatus` 还能显示启用状态、加载策略、当前模式与配置文件模式。

重要参数 / 形式

getenforce

快速返回 Enforcing、Permissive 或 Disabled。

sestatus

比较 Current mode 与 Mode from config file。

-v

在系统支持的输出中附加部分文件和进程上下文信息；不是本章默认取证入口。

setenforce

SYNOPSIS

```
setenforce {0|1|Permissive|Enforcing}
```

临时改变已启用 SELinux 的当前执行模式，不修改持久配置。需要足够权限；Disabled 不能靠该命令直接恢复为 Enforcing。

重要参数 / 形式

```
setenforce 0
```

当前切到 Permissive，用于受控诊断窗口。

```
setenforce 1
```

当前恢复 Enforcing；调查结束必须重新验证。

```
ps -Z / ls -Z
```

SYNOPSIS

```
ps -eZ
ps -Z -p PID
ls -Zd PATH
ls -lZ PATH
```

把业务进程和文件系统对象映射到当前 SELinux 上下文。`ps -Z` 观察主体 domain，`ls -Z` 观察目标 type。

重要参数 / 形式

```
ps -eZ
```

浏览系统进程及其上下文。

```
ps -Z -p PID
```

精确核对目标进程，不用命令名替代 domain。

```
ls -Zd DIR
```

查看目录对象本身，而不是目录内容。

```
ls -lZ FILE
```

同时显示 DAC 属性和文件当前上下文。

```
ausearch
```

SYNOPSIS

```
ausearch -m MESSAGE_TYPES -ts START_TIME [OPTIONS]
```

按 Audit 消息类型和时间范围检索拒绝事件。稳定顺序是记录时间、重新触发、查询近期事件，再逐步缩小。

重要参数 / 形式

`-m AVC,USER_AVC`

选择最常见的 SELinux 拒绝消息类型。

`-m AVC,USER_AVC,SELINUX_ERR,USER_SELINUX_ERR`

扩展到 SELinux 错误类消息。

`-ts recent`

从近期时间窗口开始查询；复现时间明确时可使用更精确起点。

`-i`

解释部分数字字段，适合人工阅读。

`--raw`

保留原始记录，适合交给后续解析工具。

audit2why

SYNOPSIS

```
audit2why [OPTIONS] < audit-records
```

解释原始 AVC 可能因何被策略拒绝。它提供调查线索，不负责选择最小、安全、持久的修复。

重要参数 / 形式

```
ausearch ... --raw | audit2why
```

保留原始 Audit 字段后再解释。

```
audit2why < saved-avc.log
```

对已保存的原始事件离线分析。

journalctl

SYNOPSIS

```
journalctl [OPTIONS...] [MATCHES...]
```

读取服务症状、内核消息和可选的 setroubleshoot 摘要。Journal 是旁证入口；Audit 事件仍是 AVC 的主要原始证据。

重要参数 / 形式

```
journalctl -u UNIT --since "-10 min"
```

按服务与时间查看症状。

```
journalctl -t setroubleshoot
```

在相应组件存在时读取 SELinux 摘要。

```
journalctl -k --since "-10 min"
```

查看近期内核消息，适合 Audit 链异常时继续取证。

[知识专题] DAC 允许之后，SELinux 怎样判断一次访问

SELinux 排错不应从命令清单开始，而应先建立一个足够小、能够解释日志的决策模型。面对任何拒绝，先问四个问题：谁发起访问，目标当前是什么类型，请求了什么行为，目标属于哪一种对象类别。只要能把日志还原成这四个对象，就可以继续判断它属于文件类型、端口类型、Boolean，还是根本不属于 SELinux。

主体进程 domain

+

目标对象 type

+

类别 tclass

+

行为 permission

→

Policy allow / deny

① [知识点] DAC 与 MAC 是两个独立的判断层

DAC 主要依赖 Linux 身份和对象权限，例如：

- 进程以哪个 UID、GID 运行；
- 文件 owner、group 与 mode；
- 路径各级目录是否具备搜索权限；
- 是否存在 POSIX ACL；
- 进程是否具备相关 capability。

SELinux 不替代这些机制，而是在系统中增加一层策略判断。因此：

DAC 拒绝 → SELinux 不会把这次访问变成允许

DAC 允许 → 仍需通过 SELinux 策略判断

这也是为什么 `chmod 777` 既不是安全答案，也不一定能解决服务访问问题。它最多改变 DAC；若 AVC 显示 `httpd_t` 不允许读取目标类型，SELinux 结论不会因为 mode 变为 `777` 而自动改变。

证据边界

`namei -l`、`getfacl` 或 `ls -l` 能帮助确认 DAC，但不能证明 SELinux 允许；`ls -Z` 能显示对象类型，但不能证明 DAC 允许。排错时应清楚自己正在证明哪一层。

② [知识点] 主体、目标、行为和对象类别构成最小访问请求

SELinux 的一次访问判断可压缩为：

```
source domain
+ target type
+ object class
+ requested permission
→ policy decision
```

以 Web 服务读取文件为例：

```
httpd_t
+ var_t
+ file
+ read
→ allow 或 deny
```

四个部分分别回答：

- `httpd_t`：谁发起访问；
- `var_t`：目标对象当前被策略视为什么用途；
- `file`：目标是普通文件，而不是目录、TCP socket 或其他对象；
- `read`：进程请求读取，而不是写入、执行或绑定端口。

同一个 domain 对不同 type、不同 class 或不同 permission 的结论可能完全不同，所以只说“SELinux 不允许 httpd”过于粗糙。准确表述应接近：“`httpd_t` 对 `var_t:file` 请求 `read` 时被拒绝。”

③ [知识点] domain 与 type 是 Type Enforcement 的工作接口

安全上下文的第三段统一称为 `type`。当它附着在进程上时，通常称为 `domain`；附着在文件、目录、端口等对象上时，通常称为 `type`。这不是两套独立字段，而是从主体与目标两个角度描述 Type Enforcement 对象。

例如：

上下文片段	所属对象	直观含义
<code>httpd_t</code>	httpd 进程	Web 服务进程域
<code>sshd_t</code>	sshd 进程	SSH 服务进程域
<code>httpd_sys_content_t</code>	文件/目录	适合由 httpd 读取的 Web 内容类型
<code>var_t</code>	文件/目录	<code>/var</code> 下较通用的对象类型，通常不是自定义 Web 内容的最终类型

本章只建立 domain/type 与 AVC 的阅读能力。怎样为路径建立持久文件类型规则、怎样应用规则，属于下一章。

④ [知识点] policy 决定允许集合，不等于“应用配置”

SELinux policy 是一组安全规则、类型定义、转换和约束的集合。RHEL 常见安装加载 `targeted` 策略，它重点限制被纳入策略的服务域；某些用户进程可能运行在较宽松的 `unconfined_t` 等域中，但仍然带有

SELinux 上下文。

下面三个“策略”不要混为一谈：

- 应用配置：例如 Apache 的 `DocumentRoot`；
- 网络或防火墙策略：例如 `firewalld zone`；
- SELinux policy：决定 domain 对 type/class/permission 的访问集合。

应用配置把目录改到 `/srv/exam-site`，不会自动向 SELinux policy 声明这个目录应作为 Web 内容。端口改为 82，也不会自动改变 SELinux 端口类型。下一章将处理这些显式配置入口。

⑤ [知识点] 策略允许只是局部结论，业务成功仍需其他证据

即使 SELinux 允许访问，也不能直接推出服务已达到业务终态。完整链路仍可能受到以下因素影响：

```
应用配置
→ 服务进程状态
→ DAC
→ SELinux
→ 监听与路由
→ firewalld
→ 客户端请求与业务响应
```

反过来，某次 `curl` 成功也不能证明 SELinux 持久配置正确：系统可能仍处于 Permissive，或者对象当前标签仅通过一次临时修改得到。验证时必须把当前模式、持久模式、当前上下文和最终功能分开。

[Cheatsheet] DAC 与 SELinux 都必须允许；把拒绝翻译为 `domain + type + class + permission`；policy 允许只证明 SELinux 这一层，不等于服务终态正确。

[知识专题] 从四段安全上下文识别 domain 与 type

看到 `system_u:system_r:httpd_t:s0` 时，初学者常把整串字符当作必须完整背诵的标签。更有效的方法是先识别四段结构，再根据对象类型决定哪些字段最有区分度。RHCSA 服务故障最常从第三段开始，但这并不意味着其他字段不存在或可以随意改写。

① [知识点] `user:role:type:level` 是四段结构

代表性上下文：

```
system_u:system_r:httpd_t:s0
```

可拆为：

字段	示例	本章需要掌握的判断
SELinux user	<code>system_u</code>	SELinux 身份映射，不等同于 <code>/etc/passwd</code> 中的同名 Linux 用户
role	<code>system_r</code>	角色字段；服务进程常见 <code>system_r</code>
type/domain	<code>httpd_t</code>	RHCSA 服务故障最常用的策略身份
level/range	<code>s0</code>	MLS/MCS 安全级别或范围；本章只要求识别位置与结构

SELinux user 与 Linux 用户不是同一个命名空间。Linux 用户决定 DAC 凭据；SELinux user 参与 MAC 身份映射。两者可能关联，但不能仅凭用户名字符串推断上下文。

② [知识点] 进程上下文的第三段是 domain

进程运行后，策略通常让它进入某个 domain。例如 httpd 主进程可能显示：

```
system_u:system_r:httpd_t:s0
```

其中 `httpd_t` 是后续 AVC 的 source domain。进程 domain 比可执行文件名更适合描述策略身份：多个进程都叫 `httpd`，策略判断仍以当前进程上下文为准；一个自定义程序若没有发生预期 domain transition，也可能留在其他 domain 中。

本章不展开 domain transition 规则，但应记住：命令名不等于 domain，必须用 `ps -Z` 取得当前证据。

③ [知识点] 文件和目录上下文的第三段描述对象用途

文件上下文例如：

```
system_u:object_r:httpd_sys_content_t:s0
```

这里常见 `object_r` 角色，而真正决定服务访问边界的通常仍是第三段 `httpd_sys_content_t`。比较下面两种当前类型：

```
httpd_sys_content_t → 适合作为 Web 内容的对象类型
var_t                → /var 范围中的通用类型，未必允许 httpd_t 按目标行为访问
```

文件名、扩展名和路径字符串本身不是策略判断的直接替代品。两个都叫 `index.html` 的文件，如果 type 不同，`httpd_t` 的访问结论也可能不同。

④ [知识点] level 常显示为 s0，但不能把它当作 type

RHEL 的 targeted 策略中，经常看到 `s0`，容器或 MCS 场景可能还带类别范围，例如 `s0:c123,c456`。本章不教授 MLS/MCS 配置，只训练两个边界：

1. level 是第四段，不要把其中的冒号类别误拆成新的上下文字段；

2. RHCSA 常见 Web 服务拒绝通常优先比较第三段 type/domain，而不是先修改 level。

⑤ [知识点] `unconfined_t` 仍然是 domain，不代表 SELinux 已关闭

用户 Shell 常可能运行在 `unconfined_t` 等较宽松域中。看到 `unconfined_t` 不能得出“SELinux Disabled”的结论；是否启用和当前模式仍应由 `getenforce`、`sestatus` 判断。domain 描述策略身份，模式描述策略是否执行拒绝，它们是两个维度。

⑥ [知识点] 上下文是当前对象属性，持久期望来自其他配置

`ps -Z` 和 `ls -Z` 显示对象当前携带的上下文。对于进程，这个状态随进程生命周期存在；对于文件，当前标签可能受创建位置、移动方式、临时修改或重新标记影响。

因此：

当前标签正确 $\neq$ 持久路径规则已经正确
当前进程域正确 $\neq$ 应用功能已经成功

第 29 章将把“当前标签”与“策略期望标签、持久 fcontext 规则”组成闭环。本章只负责准确观察和解读。

[Cheatsheet] 四段为 `user:role:type:level`；进程 type 称为 domain；文件 type 描述策略用途；命令名、路径名和当前标签都不能替代实际上下文证据。

[操作专题] 识别当前模式、持久模式与策略状态

模式调查的本质是分清三件事：SELinux 是否启用、当前是否执行拒绝、系统下次按配置启动时计划进入什么模式。把它们压缩成一个“SELinux 开着”会遗漏最关键的持久性差异。

① [操作] 使用 `getenforce` 快速读取当前模式

作用对象：当前运行内核中的 SELinux 状态。基本语义：输出 `Enforcing`、`Permissive` 或 `Disabled`。典型形式：

```
getenforce
```

代表性输出：

```
Enforcing
```

它适合快速回答“此刻是否执行策略拒绝”，但不显示加载的策略名，也不直接显示 `/etc/selinux/config` 中的计划模式。

② [操作] 使用 `sestatus` 同时读取状态总览

```
sestatus
```

重点字段可按下面方式解读：

```
SELinux status:          enabled
Loaded policy name:       targeted
Current mode:            enforcing
Mode from config file:    enforcing
```

- `SELinux status`：SELinux 基础设施是否启用；
- `Loaded policy name`：当前加载的策略，例如 `targeted`；
- `Current mode`：当前是否执行拒绝；
- `Mode from config file`：配置文件记录的启动模式。

`Current mode` 与 `Mode from config file` 不一致并不矛盾。例如管理员执行 `setenforce 0` 后，当前可能是 `permissive`，而配置文件仍是 `enforcing`。

边界

启动时还可能存在内核参数等覆盖因素。RHCSA 常规任务优先核对 `sestatus` 与 `/etc/selinux/config`；若出现异常启动状态，再转到第 30 章《启动链、GRUB、Target 与系统恢复》。

③ [操作] 使用 `setenforce` 进行临时模式切换

```
setenforce 0 # 当前切到 Permissive
setenforce 1 # 当前切回 Enforcing
```

作用对象：已启用 SELinux 的当前运行状态。持久性：不修改 `/etc/selinux/config`，重启后按持久配置决定。限制：`Disabled` 状态不能通过 `setenforce 1` 直接变为 `Enforcing`。

切换前后应立即取证：

```
getenforce
setenforce 0
getenforce
# 复现并收集证据
setenforce 1
getenforce
```

不要在终端中执行 `setenforce 0` 后忘记恢复。经典任务要求把“诊断窗口”与“最终安全状态”分开记录。

④ [操作] 读取 `/etc/selinux/config` 中的持久计划

```
grep -E '^[[:space:]]*SELINUX(=|TYPE=)' /etc/selinux/config
```

代表性配置：

```
SELINUX=enforcing
SELINUXTYPE=targeted
```

`SELINUX=` 记录启动模式，`SELINUXTYPE=` 选择策略类型。修改该文件不会立刻改变当前模式；当前模式仍用 `getenforce` 或 `sestatus` 验证。

本章不把 `SELINUX=disabled` 作为解决服务故障的方法。`Disabled` 会停止加载策略，并造成持久对象不被正常标记；以后重新启用还可能需重启和文件系统重新标记。这类启动级恢复边界属于第 30 章。

⑤ [知识点] Enforcing、Permissive 与 Disabled 的证据含义

状态	策略加载	对象标记	AVC	不允许的操作
Enforcing	是	正常维护	记录可见拒绝	真正拒绝
Permissive	是	继续维护	记录本应发生的拒绝	不执行拒绝
Disabled	否	不正常维护持久对象标签	无正常策略判断链	SELinux 不参与

`Permissive` 适合有限诊断，不适合作为服务最终状态。`Disabled` 比 `Permissive` 更激进，也更难安全恢复。

⑥ [诊断边界] Permissive 测试的正反结论

若切换到 `Permissive` 后目标操作成功，可以得出：

SELinux enforcement 参与了原失败路径，应收集 AVC 并判断具体策略条件。

不能得出：

已经修好，保持 `Permissive` 即可。

若切换到 `Permissive` 后同一症状仍然存在，可以得出：

当前观察到的症状不是只靠取消 SELinux 拒绝就能消失，应优先检查其他层。

也不要因此删除已有 AVC；系统可能同时存在 SELinux 偏差和另一个故障，只是后者仍然阻塞最终功能。

⑦ [边界] 单域 Permissive 不属于本章核心操作

RHEL 支持在系统保持 Enforcing 时把个别 domain 设为 permissive。这是策略开发和精细调试工具，但会降低该 domain 的保护。本章只认识这项能力，不把它作为 RHCSA 默认诊断答案，也不制作要求直接修改单域状态的核心操作题。

[验证]

```
getenforce
sestatus
grep -E '^[:space:]*SELINUX(=|TYPE=)' /etc/selinux/config
```

[Cheatsheet] `getenforce` 看当前；`sestatus` 比较当前、策略和配置模式；`setenforce` 只临时切换；持久计划在 `/etc/selinux/config`；最终应恢复题目要求的 Enforcing。

[操作专题] 用 `ps -Z` 与 `ls -Z` 建立主体和目标证据

模式只说明策略是否执行，不能解释哪个 domain 正在访问哪个 type。下一步要把业务对象与安全上下文对应起来。稳定做法是先精确识别目标进程或路径，再显示上下文，而不是把 `grep` 结果直接当作策略身份。

① [操作] 使用 `ps -eZ` 浏览进程域

```
ps -eZ
```

它在进程列表前显示 SELinux 上下文，适合建立全局视图。为了便于判断，还可以定制普通进程字段：

```
ps -e -o label,pid,ppid,user,comm,args
```

不同 `ps` 版本的默认列顺序可能不同，因此不要依赖“第三列一定是 domain”之类的位置记忆，应识别完整上下文的第三段。

② [操作] 精确查询目标 PID 的上下文

先用可靠条件得到 PID，再查询：

```
pidof httpd
ps -Z -p <PID>
```

或在已知服务主 PID 时：

```
systemctl show httpd.service -p MainPID --value
ps -Z -p "$(systemctl show httpd.service -p MainPID --value)"
```

第二种形式引用了 systemd 的前置能力，本章不重新解释 unit 状态。重点是：PID 选择必须可靠，`ps -Z` 才能成为目标进程的证据。

代表性输出可能包含：

LABEL	PID TTY	TIME CMD
system_u:system_r:httpd_t:s0	1842 ?	00:00:00 httpd

此时能证明 PID 1842 当前运行于 `httpd_t`，不能证明它一定能读取目标文件，也不能证明所有 worker 都处于同一上下文。多进程服务可按 PID 集合继续检查。

③ [操作] 使用 `ls -Zd` 查看目录本身

```
ls -Zd /srv/exam-site
```

`-Z` 显示上下文，`-d` 让 `ls` 显示目录对象本身，而不是列出目录内容。若要查看具体文件：

```
ls -lZ /srv/exam-site/index.html
```

代表性输出：

```
-rw-r--r--. root root unconfined_u:object_r:var_t:s0 /srv/exam-site/index.html
```

第一段 `-rw-r--r--.` 属于 DAC/扩展属性表现；中间的 `unconfined_u:object_r:var_t:s0` 是 SELinux 上下文。不要因为文件 owner 是 root 就忽略 `var_t`，也不要因为 type 看起来可疑就跳过普通路径权限。

④ [操作] 区分目录对象与目录内容

下面两条命令回答不同问题：

<code>ls -Zd /srv/exam-site</code>	# 目录本身的上下文
<code>ls -lZ /srv/exam-site</code>	# 目录内条目的上下文

服务访问路径时，目录搜索、文件读取和创建新对象可能分别涉及不同 class 与 permission。只检查首页文件而忽略父目录，仍可能遗漏 AVC 中对 `dir` 类的 `search` 或 `getattr` 拒绝。

⑤ [操作] 使用 `id -Z` 观察当前进程的 SELinux 上下文

```
id -Z
```

该命令常用于查看当前 Shell 所在上下文。它能帮助理解 Linux 用户与 SELinux user/domain 的区别，但不能代替对目标服务进程执行 `ps -Z`。管理员 Shell 处于 `unconfined_t`，不代表 httpd 也处于该 domain。

⑥ [验证边界] 当前上下文不等于持久配置

`ls -Z` 只能证明对象现在携带什么标签。对象可能通过临时方式被改过，也可能在重新标记、复制或重新创建后发生变化。因此本章的结论应写成：

当前文件 type 为 `var_t`，与 AVC 中的 target type 一致；下一章应检查路径的持久文件上下文规则。

而不是直接写：

文件配置错误，执行某条修复命令即可。

同样，`ps -Z` 证明当前 domain，但不证明服务配置、当前监听、开机启动或业务响应。

⑦ [帮助入口] 查询本地命令说明

```
man ps
man ls
man selinux
```

系统若提供 SELinux 专用服务手册页，也可以在第 29 章结合文件类型、端口和 Boolean 使用。这里先掌握通用观察接口。

[Cheatsheet] 全局进程域：`ps -eZ`；单 PID：`ps -Z -p`；目录本身：`ls -Zd`；具体文件：`ls -lZ`；当前上下文是证据，不是持久规则证明。

[知识专题] 把一条 AVC 还原成一次访问请求

AVC 是 Access Vector Cache 的缩写。SELinux 访问决定会被缓存，相关拒绝事件通常以 `AVC` 或 `USER_AVC` 类型进入 Audit 记录。日志看起来很长，但初步分析不需要解释每个数字；先把 source、target、class 与 permission 找出来，就能形成可验证假设。

① [知识点] 代表性 AVC 的稳定骨架

下面是为了讲解字段而整理的代表性裁剪记录，不是本会话真实执行输出：

```
type=AVC msg=audit(1770000000.123:421): avc: denied { open read } for \
pid=1842 comm="httpd" path="/srv/exam-site/index.html" \
scontext=system_u:system_r:httpd_t:s0 \
tcontext=unconfined_u:object_r:var_t:s0 \
tclass=file permissive=0
```

将它压缩成一句话：

`httpd_t` domain 对 `var_t` 类型的普通文件请求 `open/read` 时，被策略拒绝；事件发生在 `enforcing` 路径中。

这句话已经足以把下一步收缩到文件对象类型，而不是非标准端口或 Boolean。

② [知识点] { `open read` } 是被拒绝的 `permission`

`avc: denied { ... }` 中的集合描述被拒绝的具体行为。常见示例：

permission	常见直观含义	仍需结合的字段
<code>read</code>	读取内容	<code>tclass=file</code> 、目标 <code>path/type</code>
<code>open</code>	打开对象	文件类型和访问模式
<code>getattr</code>	读取元数据	目标 <code>class/type</code>
<code>search</code>	遍历目录	<code>tclass=dir</code> 、父目录上下文
<code>write</code>	修改内容	目标 <code>type</code> 与 DAC 写权限
<code>name_bind</code>	绑定网络端口	<code>tclass=tcp_socket / udp_socket</code> 与端口类型

同一业务动作可能产生多个 `permission`。不要只看到 `read` 就忽略同一事件中目录 `search` 或文件 `open` 的拒绝。

③ [知识点] `scontext` 指向发起访问的 `source context`

```
scontext=system_u:system_r:httpd_t:s0
```

第三段 `httpd_t` 是 `source domain`。将它与 `ps -Z` 对照，可以验证日志中的主体是否就是当前目标服务。若 `comm="httpd"`，但 `scontext` 不是预期 `domain`，应继续调查进程启动方式和 `domain transition`，而不是仅凭命令名下结论。

④ [知识点] `tcontext` 指向目标对象的当前 `context`

```
tcontext=unconfined_u:object_r:var_t:s0
```

第三段 `var_t` 是 `target type`。将它与 `ls -Z` 对照，可以确认当前对象是否仍带有日志记录中的类型。目标可能在调查期间被改动，所以日志证据与当前状态都要保留时间关系。

⑤ [知识点] `tclass` 决定“目标是什么对象”

`tclass=file` 表示普通文件；还可能看到：

- `dir`：目录；

- `tcp_socket`、`udp_socket`：网络 socket；
- `lnk_file`：符号链接；
- `process`：进程对象；
- `sock_file`：Unix socket 文件。

同一个 permission 名称在不同 class 中可能含义不同。诊断时应把 `tclass` 与 permission 成对阅读。

⑥ [知识点] `comm`、`exe`、PID 和路径用于关联业务动作

- `comm`：进程短名称，可能被长度限制；
- `pid`：事件发生时的进程实例，之后可能退出或被复用；
- `exe`：完整可执行路径常位于同一 Audit 事件的其他记录中；
- `path / name`：目标路径可能位于 AVC 或配套 `PATH` 记录中。

因此，一条完整 Audit 事件可能由多条记录组成。它们通过相同的 `msg=audit(时间:序号)` 关联。`ausearch` 会按事件组织输出，比直接 `grep audit.log` 更适合初步调查。

⑦ [知识点] `permissive=0` 与 `permissive=1` 说明执行边界

- `permissive=0`：该拒绝发生在执行策略拒绝的路径；
- `permissive=1`：策略会拒绝，但当前处于允许通过的 `permissive` 路径。

在系统级 `Permissive` 或单域 `Permissive` 中，业务动作可能继续执行并触发更多后续 AVC。因此，`Permissive` 中看到的拒绝序列可能比 `Enforcing` 更长；不能把每条后续 AVC 都机械视为原始失败的第一根因。

⑧ [知识点] 没有 AVC 不等于 SELinux 一定未参与

常见原因包括：

- 没有重新触发失败；
- 查询时间范围不对；
- `auditd` 没有运行；
- 记录进入 Journal 或内核消息；
- `dontaudit` 规则抑制了某些拒绝；
- 当前症状由其他层造成。

因此“无结果”也是一个需要继续验证的状态，而不是立即关闭 SELinux 的理由。

[Cheatsheet] 先读 `{ permission }`、`scontext`、`tcontext`、`tclass`；再用 `comm`、PID、路径和事件序号关联业务；把长日志翻译成一句 `domain 对 type:class 请求 permission`。

[操作专题] 重新触发、检索并解释近期拒绝

日志调查最常见的失败不是不会写 `ausearch`，而是没有建立时间边界：管理员查询几小时的旧记录，却不知道哪条属于当前复现。推荐顺序是记录当前模式与时间，清楚地重新触发一次目标动作，然后立即查询 `recent` 或明确时间范围。

① [操作] 先建立复现时间与当前模式

```
date --iso-8601=seconds  
getenforce
```

随后执行一次最小、可重复的目标操作，例如：

```
curl -sS -o /dev/null -w '%{http_code}\n' http://127.0.0.1/
```

这里的 `curl` 只是代表业务触发入口。真实服务可能通过文件访问、启动服务或绑定端口触发拒绝。复现前不要一次改动多个配置，否则新日志无法对应单一假设。

② [操作] 使用完整消息类型集合搜索近期拒绝

官方排错入口：

```
ausearch \  
-m AVC,USER_AVC,SELINUX_ERR,USER_SELINUX_ERR \  
-ts recent
```

为了人工阅读，可增加解释选项：

```
ausearch \  
-m AVC,USER_AVC,SELINUX_ERR,USER_SELINUX_ERR \  
-ts recent \  
-i
```

`-m` 选择消息类型，`-ts` 限制开始时间，`-i` 将部分数字字段解释为更易读的名称。若只训练最常见的核心入口，也可以先使用：

```
ausearch -m AVC,USER_AVC -ts recent -i
```

③ [参数] `-i` 面向人工阅读，`--raw` 面向后续工具

`-i` 会解释字段，适合直接阅读；`--raw` 保留原始 Audit 记录，适合通过管道交给解析工具。不要把两者混成同一个目的。

人工阅读：

```
ausearch -m AVC,USER_AVC -ts recent -i
```

交给 `audit2why` :

```
ausearch -m AVC,USER_AVC -ts recent --raw | audit2why
```

④ [操作] 根据已知对象继续缩小结果

`ausearch` 的多个筛选条件通常按 AND 组合。已知命令名、PID 或可执行路径时，可以在本地 `man ausearch` 中选择相应参数继续缩小。考试和工作中应先保留一份未过度过滤的近期事件，再增加条件，避免因命令名截断或 PID 已变化而错过记录。

稳定的最小路径仍是：

```
近期消息类型
→ 识别事件序号
→ 读取完整事件
→ 再按业务对象缩小
```

⑤ [操作] 用 `journalctl` 查服务症状与 `setroubleshoot` 摘要

服务侧症状：

```
journalctl -u httpd.service --since "-10 min"
```

若系统安装并运行了相应 `setroubleshoot` 组件，可查询：

```
journalctl -t setroubleshoot
```

Journal 摘要便于阅读，但 Audit 事件仍是 SELinux 拒绝的主要原始证据。没有 `setroubleshoot` 输出不能证明不存在 AVC。

⑥ [操作] `auditd` 未运行时的后备入口

先确认：

```
systemctl is-active auditd
```

若 Audit daemon 未运行，应按环境要求恢复服务并重新触发。SELinux 活跃但 Audit daemon 不工作时，官方资料还建议从内核消息中搜索特定类型：

```
dmesg | grep -i -e 'type=1300' -e 'type=1400'
```

本章不展开 Audit 规则配置；目标只是避免把“Audit 文件无记录”误写成“SELinux 未参与”。

⑦ [操作] 使用 `audit2why` 辅助解释

```
ausearch -m AVC,USER_AVC -ts recent --raw | audit2why
```

`audit2why` 可能帮助指出约束、Boolean 或缺少允许规则等方向。它的输出应被视为线索，仍要与以下事实交叉核对：

- 当前进程 domain；
- 当前对象 type；
- 业务目标是否合理；
- 标准策略是否已有文件类型、端口类型或 Boolean；
- 建议是否扩大了不必要的权限。

⑧ [安全边界] 不把 `audit2allow` 当作第一修复手段

自动从拒绝生成允许规则，可能把误配置包装成新的权限。常见拒绝首先应检查：

```
错误或非预期标签
非标准目录未声明
非标准端口未映射
业务行为需要已有 Boolean
应用或 DAC 本身配置错误
```

只有确认标准策略无法表达合法需求时，才进入自定义策略模块；该能力不属于本章，也不是 RHCSA 常规服务题的默认答案。

⑨ [边界] `dontaudit` 调试必须成对恢复

极少数拒绝可能被 `dontaudit` 抑制。官方排错资料提供临时禁用和恢复方式：

```
semodule -DB # 临时禁用 dontaudit 规则后重新复现
semodule -B # 调查结束后恢复
```

这是会改变策略日志行为的高级诊断步骤，不是每次无 AVC 都应执行的默认命令。必须先检查复现、时间范围、Audit daemon 和 Journal，并确保调查后恢复。

[Cheatsheet] 记 录 时 间 并 复 现 ；
`ausearch -m AVC,USER_AVC,SELINUX_ERR,USER_SELINUX_ERR -ts recent`；人工读用 `-i`，管道解析用 `--raw`；Journal 是辅助；`audit2why` 是线索，不是修复判决。

[诊断专题] 先证据后修复，并正确使用 Permissive

SELinux 排错的目标不是尽快让错误消失，而是找到最小、可解释、可持久验证的配置入口。下面的诊断链把每一步都写成一个可证伪的假设。已有可靠证据时可以从中间切入，但不能跳过结论所依赖的状态。

① [诊断] 从症状建立多层假设，而不是直接归因

症状：“httpd 访问自定义目录返回 403。”

初步假设至少包括：

- Apache 配置或目录授权规则拒绝；
- 服务读取的并不是预期文件；
- DAC 路径权限不允许；
- SELinux domain/type 组合被拒绝；
- 请求经过代理、虚拟主机或其他网络路径。

先用服务日志、配置检查、路径权限和请求结果建立基线，再把 SELinux 作为其中一层调查。

② [诊断] 下一条最有区分度的证据通常是模式和近期 AVC

若系统当前是 Permissive，业务成功不能证明策略允许；若当前是 Enforcing，业务失败也不能证明 SELinux 是根因。最有区分度的组合是：

```
getenforce
# 重新触发一次目标操作
ausearch -m AVC,USER_AVC -ts recent -i
```

- Enforcing + 有匹配 AVC：SELinux 明确拒绝了某个请求；
- Enforcing + 无匹配 AVC：检查 Audit/Journal、时间和其他层；
- Permissive + 有 AVC + 业务成功：策略原本会拒绝，但当前未执行；
- Permissive + 无 AVC：不能凭空编造 SELinux 根因。

③ [诊断] 用上下文证据验证 AVC 中的主体和目标

```
ps -Z -p <PID>
ls -Zd <DIRECTORY>
ls -lZ <FILE>
```

若当前状态与日志一致，可以继续分类；若不一致，应先回答对象是否在记录后被修改、进程是否重启、PID 是否复用、请求是否命中了另一个路径。

④ [诊断] 把 AVC 分类到标准策略入口

AVC 特征	典型分类	下一章的自然入口
<code>tclass=file/dir</code> ，目标 type 不符合服务用途	文件上下文	持久 fcontext 规则与 <code>restorecon</code>
<code>denied { name_bind }</code> ， <code>tclass=tcp_socket</code>	端口类型	<code>semanage port</code>
domain/type 看似正常，行为属于策略预置可选能力	Boolean	<code>getsebool</code> / <code>setsebool -P</code>
建议缺少规则，但标准入口均不适用	可能需要自定义策略	超出 RHCSA 本章，先复核需求
Permissive 后仍是相同业务错误且无相关 AVC	非 SELinux 主阻塞	回到应用、DAC、网络或数据层

本章只做到“有证据地分类”，不执行前三类的完整修复。

⑤ [诊断] Permissive 是实验变量，不是最终答案

一个可审计的 Permissive 诊断窗口应记录：

1. 切换前模式；
2. 目标操作和时间；
3. 切换后是否成功；
4. 新增 AVC；
5. 调查结束后恢复 Enforcing；
6. 当前和持久模式的最终证据。

仅执行 `setenforce 0` 并说“现在能用了”，既没有修复，也没有保留原因。

⑥ [诊断] 最小修复必须留给正确对象

AVC 指向 `var_t:file` 读取拒绝时，最小修复对象通常是路径的 SELinux 文件类型，而不是：

- 把整个目录改成 `777`；
- 开启所有 httpd Boolean；
- 新增任意自定义 allow 规则；
- 永久保持 Permissive；
- 关闭 SELinux。

本章应输出交接结论：

当前证据指向自定义路径文件类型；下一章检查并建立持久文件上下文规则，然后重新验证当前标签和业务。

⑦ [验证] 诊断完成不等于业务修复完成

本章结束时可以形成一个完整的诊断终态：

- 当前与持久模式已明确；
- 进程 domain 与对象 type 已明确；
- AVC 中的 class 与 permission 已明确；
- 拒绝被分类到正确配置入口；
- 系统恢复到题目要求的安全模式；
- 未声称尚未执行的第 29 章修复已经完成。

这比“服务 active”或“命令返回 0”更可信，因为每个结论都有独立证据。

⑧ [工作迁移] 为变更单保留最小证据包

真实工作中，可保留：

故障时间与复现步骤
当前/持久 SELinux 模式
目标进程 PID、命令行和上下文
目标路径及当前上下文
相关 Audit 事件原文
一句话的 AVC 翻译
分类与建议的标准配置入口
变更后需要重新验证的层次

不要把整份 `/var/log/audit/audit.log` 无差别复制进报告，也不要只保留工具生成的结论而丢失原始 AVC。

[Cheatsheet] 症状 → 模式与复现 → AVC → `ps -Z / ls -Z` → 四元翻译 → 分类标准入口 → 最小修复 → 恢复 Enforcing → 分层再验证。

[经典任务] 在不关闭 SELinux 的前提下完成自定义 Web 目录 AVC 取证

一台 RHEL 9 服务器已经安装并启动 `httpd`。管理员把站点目录改为 `/srv/exam-site`，目录中已有 `/srv/exam-site/index.html`，不得删除、覆盖或改写该文件。

已知环境：

- Apache 仍监听标准 TCP 80，本任务不处理非标准端口；
- 普通目录搜索和文件读取权限已经满足；
- `firewalld` 不是本机回环请求的阻塞点；
- SELinux 当前为 `Permissive`；
- `/etc/selinux/config` 计划下次启动进入 `Enforcing`；
- 当前通过 `curl http://127.0.0.1/` 可以读取页面；
- Audit 日志中应能找到本次访问对应的 AVC；
- 当前没有可由本会话控制的真实实验机，因此题目中的命令输出均需在你的 RHEL 9 环境中取得，本参考解答只给出推荐调查路径。

请完成以下目标：

1. 证明当前模式和持久配置原本不一致；
2. 确认 `httpd` 主进程当前运行在哪个 SELinux domain；
3. 确认 `/srv/exam-site` 目录和 `index.html` 当前是什么 type；
4. 记录时间并重新触发一次 HTTP 请求；
5. 使用 `ausearch` 提取近期 AVC，并保留一份适合人工阅读的结果；
6. 从事件中提炼 source domain、target type、object class 和 denied permission；
7. 使用原始事件通过 `audit2why` 做辅助解释；
8. 判断下一步应进入“持久文件上下文规则”调查，而不是端口类型或 Boolean；
9. 本章内不得执行 `semanage fcontext`、`restorecon`、`chcon`、`semanage port` 或 `setsebool`；
10. 调查结束后将当前模式恢复为 `Enforcing`，并证明当前模式与持久配置均为 `Enforcing`；
11. 明确记录：在第 29 章修复文件规则之前，`Enforcing` 下页面仍可能访问失败，这不影响本任务的诊断终态成立。

限制条件：

不得关闭 SELinux
不得把 Permissive 作为最终状态
不得 chmod 777
不得删除或替换现有页面
不得使用 audit2allow 生成策略模块
不得把“页面在 Permissive 下可访问”写成最终修复

验收矩阵

目标	推荐证据	能证明什么
当前模式	<code>getenforce</code>	当前是否执行拒绝
当前与配置模式	<code>sestatus</code>	两个模式维度与策略名
持久配置	<code>grep ... /etc/selinux/config</code>	配置文件记录
进程域	<code>ps -Z -p <PID></code>	当前 httpd domain
目录/文件类型	<code>ls -Zd</code> 、 <code>ls -lZ</code>	当前对象上下文
近期拒绝	<code>ausearch ... -ts recent -i</code>	人工可读的 SELinux 事件
辅助解释	<code>ausearch ... --raw audit2why</code>	原因线索
最终安全模式	再次执行 <code>getenforce</code> 、 <code>sestatus</code>	已恢复 Enforcing
诊断交接	一句四元翻译与分类	下一章应处理的对象

卡住时再想一想

页面在 Permissive 下成功，证明的是策略允许，还是只证明拒绝没有执行？

`comm="httpd"` 能否替代 `scontext` 判断 domain？

`ls -Z` 显示当前 type 后，是否已经证明持久路径规则存在？

请先独立完成调查。参考解答从下一页开始。

[参考解答] 从基线、复现到 AVC 分类

① 建立模式基线

```
getenforce
sestatus
grep -E '^[:space:]]*SELINUX(=|TYPE=)' /etc/selinux/config
```

预期要识别的关系是：

```
Current mode: permissive
Mode from config file: enforcing
```

或 `getenforce` 返回 `Permissive`，配置文件记录 `SELINUX=enforcing`。这说明当前诊断状态不会跨重启自动保持，并且当前业务成功不能证明策略允许。

不要立即执行 `setenforce 1`。题目要求先在可重复的诊断窗口中收集 AVC，完成取证后再恢复。

② 精确取得 httpd 主进程和 domain

```
main_pid=$(systemctl show httpd.service -p MainPID --value)
printf 'MainPID=%s\n' "$main_pid"
ps -Z -p "$main_pid"
```

若 `MainPID` 为 `0`，应先回到服务状态调查，而不是继续假定进程存在：

```
systemctl status httpd.service
journalctl -u httpd.service --since "-10 min"
```

目标是得到类似 `httpd_t` 的当前 domain。不要只运行 `ps aux | grep httpd`，因为它没有显示 SELinux 上下文，还可能包含 `grep` 本身或多个 worker。

③ 取得目录和文件当前 type

```
ls -Zd /srv/exam-site
ls -lZ /srv/exam-site/index.html
```

记录完整四段上下文，并单独提取第三段 type。若看到 `var_t` 或其他不符合 Web 内容用途的类型，只形成假设，不在本章直接修复。

同时保留 DAC 基线，证明题目给定条件仍成立：

```
namei -l /srv/exam-site/index.html
```

④ 记录时间并重新触发一次请求

```
date --iso-8601=seconds  
curl -sS -D - http://127.0.0.1/ -o /dev/null
```

在 Permissive 下请求可能成功，但此时策略仍会记录本应发生的拒绝。只触发一次有助于把近期事件与操作关联。

⑤ 提取人工可读 AVC

```
ausearch \  
-m AVC,USER_AVC,SELINUX_ERR,USER_SELINUX_ERR \  
-ts recent \  
-i
```

若结果较多，先用事件时间和 `comm` 找到与刚才请求相符的事件，再保留同一事件序号的完整记录。不要只复制单独一行而丢失配套 `PATH` 或 `SYSCALL` 记录。

若没有结果，按顺序调查：

```
systemctl is-active auditd  
journalctl -t setroubleshoot --since "-10 min"  
dmesg | grep -i -e 'type=1300' -e 'type=1400'
```

先确认复现和日志链，不要直接关闭 SELinux 或生成策略。

⑥ 用四元模型翻译事件

假设代表性 AVC 包含：

```
avc: denied { open read }  
scontext=...:httpd_t:s0  
tcontext=...:var_t:s0  
tclass=file  
permissive=1
```

应写成：

```
httpd_t domain 对 var_t:file 请求 open/read；策略会拒绝，但当前 permissive 路径未执行拒绝。
```

再与当前状态交叉核对：

```
ps -Z -p "$main_pid"
ls -lZ /srv/exam-site/index.html
```

若日志和当前 type 不一致，先调查对象是否已被修改或请求是否访问了其他文件。

⑦ 通过原始事件调用 `audit2why`

```
ausearch \
  -m AVC,USER_AVC \
  -ts recent \
  --raw | audit2why
```

将输出作为辅助线索。即使工具提到可能生成允许规则，也要先检查标准策略入口。当前 `tclass=file`、目标 type 为 `var_t`，而业务是自定义 Web 内容，最合理的分类是文件上下文问题。

⑧ 形成交接结论，不越界执行修复

推荐结论：

证据显示 `httpd_t` 对 `/srv/exam-site/index.html` 的当前 `var_t:file` 请求 `open/read` 时产生 `AVC`。目标为文件对象，不是 `tcp_socket`；当前任务也没有数据库外连等 `Boolean` 需求。下一步应在第 29 章检查该路径的持久文件上下文规则，再按规则恢复当前标签并重新验证。

这里不得使用 `chcon` 制造临时正确标签，也不得提前执行第 29 章操作。

⑨ 恢复 Enforcing 并验证模式终态

```
setenforce 1
getenforce
sestatus
grep -E '^[:space:]*SELINUX(=|TYPE=)' /etc/selinux/config
```

目标是：当前为 Enforcing，配置文件也计划 Enforcing。

可以再次请求页面以观察 Enforcing 下的真实表现：

```
curl -sS -D - http://127.0.0.1/ -o /dev/null
```

若访问失败，这与诊断结论一致：本章已经完成证据收集和分类，但尚未执行文件规则修复。不要把“页面仍失败”误判为本任务失败，也不要把它隐藏在报告中。

⑩ 分层验收与典型错误

模式层：当前与持久均为 Enforcing。主体层：目标 httpd 进程 domain 已记录。目标层：目录和文件当前 type 已记录。事件层：近期 AVC 原文、事件序号和四元翻译已保留。诊断层：已分类到文件上下文，不误归因到端口或 Boolean。边界层：未关闭 SELinux、未放宽 DAC、未生成本地策略、未抢先执行第 29 章修复。

典型错误：

- 1. 看到 curl 在 Permissive 下成功，写成“SELinux 已配置完成”；
- 2. 只保存 audit2why 文本，丢失原始 AVC；
- 3. 只看 comm="httpd"，不核对 scontext；
- 4. 看到 ls -Z 后直接宣称持久规则存在；
- 5. 调查结束忘记 setenforce 1；
- 6. 用 audit2allow 为错误标签生成额外权限。

[本章收束] 从拒绝表象回到可验证的策略对象

工作方法：把每一次拒绝写成一条可核对的判断句

本章最终训练的不是“记住几条 SELinux 命令”，而是把故障还原为三张状态图。

访问决策: source domain + target type + object class + permission → policy decision
模式关系: 当前模式 ≠ 持久计划
证据推进: 症状 → 复现 → AVC → 上下文核对 → 分类标准入口 → 再验证

真正可交接的诊断结论应包含：当前模式、复现时间、主体 domain、目标 type、对象类别、被拒绝权限、原始事件位置，以及为什么下一步应进入某个标准配置入口。只写“SELinux 拒绝了”仍然不够精确。

主要判断表

已取得的证据	可以得出的结论	不能扩大的结论	下一条有区分度的证据
getenforce 为 Enforcing	当前执行策略拒绝	当前故障一定由 SELinux 引起	重新触发并查询近期 AVC
当前为 Permissive 且存在 AVC	策略本应拒绝，但当前未执行拒绝	当前配置已经正确	解读四元组并恢复 Enforcing 验证
ps -Z 显示 httpd_t	目标进程当前处于该 domain	它一定能访问目标对象	ls -Z 与 AVC 的 tcontext
ls -Z 显示 var_t	对象当前 type 为 var_t	持久 fcontext 规则一定错误或不存在	第 29 章的期望类型与持久规则查询
AVC 为 tclass=file、read	文件读取路径被策略拒绝	端口或 Boolean 一定无问题	核对 path、当前 type 和业务目标

已取得的证据	可以得出的结论	不能扩大的结论	下一条有区分度的证据
<code>audit2why</code> 给出建议	获得一个解释方向	建议就是最小、安全修复	回到原始 AVC 和标准策略入口
Permissive 后仍失败	取消 SELinux 拒绝不足以消除当前症状	SELinux 完全没有任何偏差	服务日志、DAC、配置、网络等其他层

向下一章交接

本章完成到“分类标准配置入口”为止。第 29 章《SELinux 文件规则、端口类型与 Boolean》将继续回答三类问题：怎样为路径建立持久文件上下文规则，怎样让服务使用策略认可的端口类型，以及怎样判断并持久设置已有 Boolean。进入下一章前，应已经能够把一条 AVC 翻译成 domain 对 type:class 请求 permission，并知道当前标签、持久规则与业务终态不是同一件事。

最终安全边界：不通过 `chmod 777`、Permissive、Disabled 或未经分析的本地 allow 规则绕过问题；不把“有 AVC”扩大成“所有故障都由 SELinux 引起”；不把工具建议当作最终判决；每一次修改都要能回到原始事件和目标业务解释。